

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

WEEK 3:

SERIAL COMMUNICATION BETWEEN ARDUINO AND PHYTON

SECTION 2

GROUP 17

LECTURER: ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO

Date of Experiment: Monday, 20 October 2025

Date of Submission: Monday, 27 October 2025

NO.	GROUP MEMBERS	MATRIC NO.
1.	MUHAMMAD IMAN FARIHIN BIN MOHD IMRAN	2313813
2.	MUHAMMAD SYAZANI AKMAL BIN MOHD NASARUDIN	2314517
3.	AININ SOFIYA BINTI KAMARUZAMAN	2314916

ABSTRACT

This experiment demonstrates the control of a servo motor using serial communication between an Arduino microcontroller and a Python program. Through this setup, the user can send commands from a computer to adjust the servo's angular position in real time. The experiment also explores how data from sensors, such as a potentiometer, can be transmitted to the computer for visualization and monitoring. By integrating hardware control and software-based data processing, the experiment highlights the fundamentals of mechatronic system integration. Specifically, how embedded systems communicate with external software interfaces. The resulting system successfully enables bidirectional control and monitoring, showing how serial communication can synchronize hardware motion with user inputs and real-time data visualization.

TABLE OF CONTENTS

INTRODUCTION

EXPERIMENT 1 & TASK 1

MATERIALS AND EQUIPMENT

EXPERIMENTAL SETUP

METHODOLOGY

RESULTS

EXPERIMENT 2 & TASK 2

MATERIALS AND EQUIPMENT

EXPERIMENTAL SETUP

METHODOLOGY

RESULTS

DISCUSSION

CONCLUSION

RECOMMENDATIONS

REFERENCES

APPENDICES

ACKNOWLEDGEMENT

INTRODUCTION

Servo motors are essential components in mechatronic systems where precise position control is required, such as in robotics, automation, and actuation mechanisms. They operate by receiving a pulse-width modulated (PWM) signal from a microcontroller, which determines the shaft angle of the motor. In this experiment, an Arduino board acts as the controller responsible for generating the PWM signal to drive the servo motor, while Python software running on a computer serves as the interface for user input and data visualization.

The communication between Arduino and Python is established through a serial connection (UART via USB), enabling data exchange between hardware and software. This allows users to send angle values from Python to the Arduino, which then moves the servo to the corresponding position. In the enhanced part of the experiment, a potentiometer is integrated to provide real-time feedback, allowing continuous adjustment of the servo angle based on the potentiometer's analog input. Additionally, the experiment demonstrates the use of matplotlib in Python for real-time graphical visualization of sensor data, strengthening the link between physical signals and digital representation.

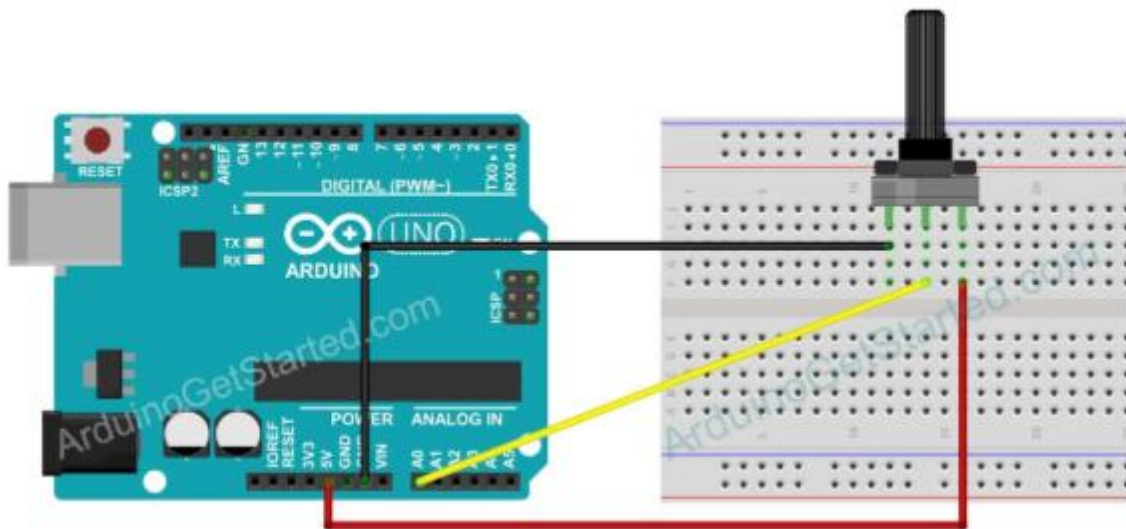
Overall, this experiment introduces key concepts in mechatronics system integration, including actuator control, sensor interfacing, and serial data communication. It builds foundational understanding for designing interactive control systems where software can dynamically manipulate and monitor physical components.

EXPERIMENT 1 & TASK 1

MATERIALS AND EQUIPMENTS

1. Arduino Board
2. Potentiometer
3. Jumper Wires
4. LED
5. 220 resistor
6. Breadboard

EXPERIMENTAL SETUP



1. One leg of the potentiometer was connected to the 5V terminal on the Arduino.
2. The other leg of the potentiometer was connected to the GND terminal on the Arduino.
3. The middle leg (wiper) of the potentiometer was connected to an analog input pin on the Arduino, specifically A0.

METHODOLOGY

1. An LED was connected to digital pin 9 of the microcontroller.
2. A potentiometer was connected to analog pin A1 to measure its variable resistance.
3. Proper wiring was ensured for the potentiometer:
 - a. One end was connected to VCC (5V).
 - b. The other end was connected to Ground (GND).
 - c. The middle terminal (wiper) was connected to analog input A0.

● Programming Logic

1. Serial communication was initiated at a baud rate of 9600 to facilitate real-time monitoring of output values.
2. Pin 9 was configured as an output for the LED
3. Reading Potentiometer Input
 - Continuously read the analog value from pin A0.
 - Convert the 0-1023 analog reading to a 0-255 range.
4. Adjusting LED Brightness
 - Use Pulse Width Modulation (PWM) to control LED brightness based on the mapped potentiometer value.
 - Output the PWM signal to pin 9.
5. Displaying Data
 - Print the brightness value to the Serial Monitor for real-time monitoring.
6. Adding Delay for Smoothness
 - Introduce a 100ms delay to prevent rapid fluctuations and ensure smooth LED brightness transitions.

● Code used

ARDUINO

```
int potPin = A0;           // select the input pin for the potentiometer
int ledPin = 2;            // select the pin for the LED
int potValue = 0;          // variable to store the value coming from the sensor

void setup() {
  pinMode(ledPin, OUTPUT); // declare the ledPin as an OUTPUT
  Serial.begin(9600);       // start serial communication
}
```

```

void loop() {
    potValue = analogRead(potPin); // read the value from the potentiometer
    Serial.println(potValue);      // print the value for debugging

    if (potValue > 512) {
        digitalWrite(ledPin, HIGH); // turn LED on if in upper half
    } else {
        digitalWrite(ledPin, LOW);  // turn LED off if in lower half
    }

    delay(100); // small delay to stabilize output
}

```

PHYTON

```

import serial
import matplotlib.pyplot as plt
import numpy as np
import time

ser = serial.Serial('COM6', 9600)

xpoints = []
ypoints = []

try:
    while True:
        pot_value = ser.readline().decode().strip()

        try:
            pot_value = int(pot_value) # convert to integer value
            print("POT Value:", pot_value)

            xpoints.append(time.time())
            ypoints.append(pot_value)

            if len(xpoints) > 100:
                xpoints.pop(0)
                ypoints.pop(0)

            # plotting
            plt.clf()
            plt.plot(xpoints, ypoints)
            plt.xlabel('Time (s)')
            plt.ylabel('Potentiometer Value')
            plt.title('Real-time Potentiometer Value')
            plt.pause(0.1)

```

```
        except ValueError:
            print('Invalid pot value received')

except KeyboardInterrupt:
    ser.close()
    print("Serial connection closed.")
except UnicodeDecodeError:
    print('Serial Error\n')
finally:
    ser.close()
    print("Serial connection closed.")
```

Control algorithm

1. Start Initialization

- Define LED_PIN as 2 (PWM output for LED).
- Define POT_PIN as A0 (analog input for potentiometer).
- Initialize serial communication at 9600 baud for monitoring.
- Set LED_PIN as an output.

2. Continuous Loop Execution

- Read the potentiometer value from POT_PIN (A0).
- Map the analog input (0-1023) to a PWM range (0-255).
- Write the mapped value to LED_PIN using analogWrite(), adjusting the LED brightness.
- Print the brightness value to the serial monitor.
- Introduce a 100ms delay for smooth transitions.

3. Repeat Indefinitely

- The loop continuously adjusts the LED brightness based on real-time potentiometer input.

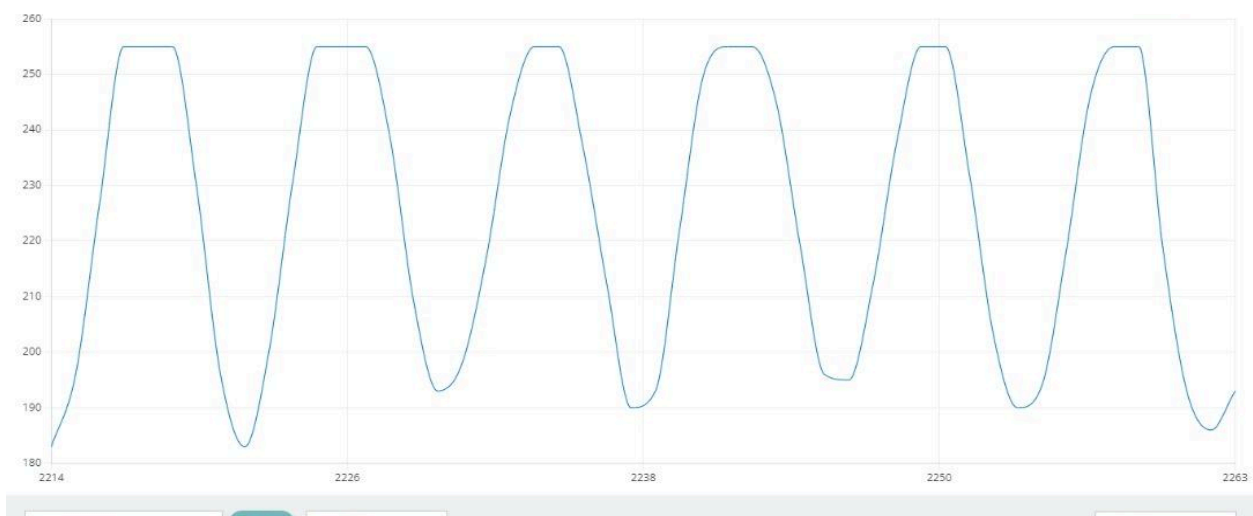
RESULT

The successful execution of the experiment demonstrates that real-time data transmission from the Arduino to Python is achievable. The plotted graph accurately represents potentiometer adjustments, validating the effectiveness of the serial communication process.

To present the potentiometer readings graphically using Python, the matplotlib library was utilized to plot real-time data retrieved from the Arduino. A step-by-step guide and the required Python script are provided below:

1. The necessary libraries, matplotlib and pyserial, were installed using the pipinstall command.
2. The Arduino was connected to the computer, and verification of the correct COM port was ensured.
3. The Arduino code was set up to continuously transmit potentiometer values via the `serial.println()` function.
4. A Python script was executed to read the incoming serial data and display it graphically.

A real-time graph was displayed upon script execution, and it was configured to represent the continuously changing potentiometer values. The graph was updated every 100ms, which resulted in a smooth visualization of the value transitions.

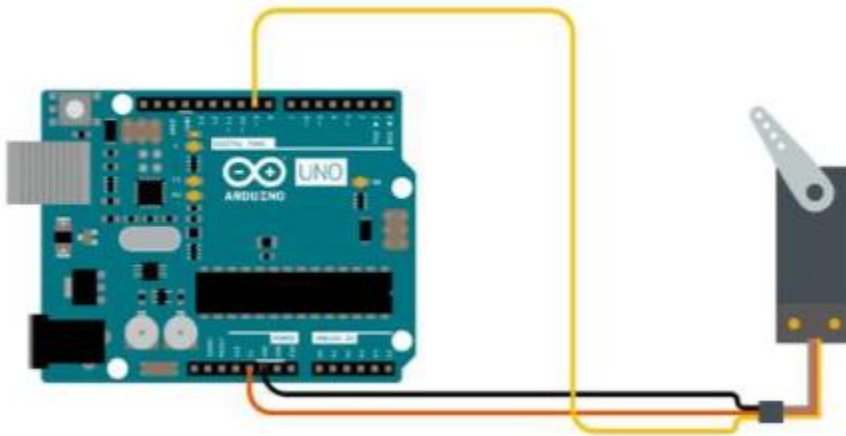


EXPERIMENT 2 & TASK 2

MATERIALS AND EQUIPMENTS

1. Arduino board (Arduino Uno)
2. Servo motor
3. Jumper wires
4. Potentiometer (for manual angle input)
5. USB cable for Arduino
6. Computer with Arduino IDE and Python installed

EXPERIMENTAL SETUP



1. The servo's signal wire was connected to a Pulse Width Modulation (PWM)-capable digital pin on the Arduino, specifically Pin 9.
2. The servo was powered using the Arduino's 5v and GND pins.
3. The servo's power wire (typically red) was connected to the 5v output on the Arduino board, as servos generally require a supply voltage of 5v.
4. The servo's ground wire (typically brown) was connected to one of the Ground (GND) pins on the Arduino.

METHODOLOGY

1. Circuit assembly

1. The potentiometer was connected to analog pin A0 to allow its variable resistance to be read. For proper configuration:
 - One end of the potentiometer was connected to VCC 5v.
 - The other end was connected to GND.
 - The middle terminal (wiper) was connected to analog pin A0.
2. The servo motor was connected to digital pin 8 of the microcontroller.
 - The signal wire was specifically routed to pin 8.
 - The power wire was connected to the 5v output terminal.
 - The ground wire was connected to the Ground (GND) terminal.

2. Programming logic

1. Attach the servo motor to pin 8 using `myServo.attach(8)`.
2. Begin serial communication at 9600 baud rate to monitor values.
3. Reading Potentiometer Input
 - Continuously read the analog value from pin A0.
 - Convert the 0-1023 analog reading to a 0-180 degree range using `map()`.
4. Controlling the Servo Motor
 - Move the servo motor to the mapped angle using `myServo.write(angle)`.
 - Print the potentiometer value and servo angle to the Serial Monitor for real-time tracking.
5. Adding Delay for Stability
 - Introduce a 10ms delay to ensure smooth movement without abrupt changes.

3. Code used

ARDUINO

```
#include <Servo.h>
```

```
Servo myservo;
```

```
int potPin = A0;
```

```
int ledPin = 2;
```

```
void setup() {
```

```
  Serial.begin(9600);
```

```
  myservo.attach(8);
```

```
  pinMode(ledPin, OUTPUT);
```

```

}

void loop() {
    int potValue = analogRead(potPin);           // read potentiometer
                                                    (0-1023)
    int angle = map(potValue, 0, 1023, 0, 180); // convert to servo angle
    myservo.write(angle);

    // LED brightness or ON/OFF based on position
    if (angle > 90) digitalWrite(ledPin, HIGH);
    else digitalWrite(ledPin, LOW);

    // send angle to Python
    Serial.println(angle);
    delay(50); // smooth output
}

```

PHYTON

```

import serial
import matplotlib.pyplot as plt
import time

# Replace 'COMx' with your actual port (e.g., 'COM6')
ser = serial.Serial('COM6', 9600)
time.sleep(2) # Wait for Arduino to reset

plt.ion() # Interactive mode
fig, ax = plt.subplots()
x_vals, y_vals = [], []

try:
    while True:
        if ser.in_waiting:
            pot_value = ser.readline().decode().strip()
            if pot_value.isdigit():
                angle = int(pot_value)
                print("Servo Angle:", angle)
                x_vals.append(len(x_vals))
                y_vals.append(angle)
                ax.clear()
                ax.set_ylim(0, 180)
                ax.plot(x_vals, y_vals, label="Servo Angle")
                ax.set_title("Real-Time Servo Angle from Potentiometer")
                ax.set_xlabel("Time Step")

```

```

        ax.set_ylabel("Angle (°)")
        ax.legend()
        plt.pause(0.05)

except KeyboardInterrupt:
    print("\n Interrupted by user.")

finally:
    ser.close()
    plt.ioff()
    plt.show()
    print(" Serial connection closed.")

```

RESULT

The servo motor successfully followed the given control signals, rotating to the intended angles with a consistent and stable motion, indicating that the motor responded correctly to the input commands.

Throughout the experiment, the servo maintained a steady performance without sudden jumps or irregular movements. The response time was relatively fast, and the motor accurately reached the target angles within a short delay. The graphical data representation showed a repetitive and smooth waveform, confirming the servo's ability to operate as expected under the given conditions.

TASK 2: IMPLEMENTATION AND ENHANCEMENT

Following verification of the foundational servo motor operation, the experiment proceeded to an enhancement phase (Task 2) focused on establishing a real-time, multi-output control system coupled with serial data visualization. This required the integration of a potentiometer and an external LED, along with the development of host-side Python scripting.

The Arduino sketch was modified to read the analog input from the potentiometer. This raw analog reading served as the control input for two simultaneous output functions:

- **Servo Angle Control:** The analog value was linearly mapped to the servo motor's full operational range (0-180 degrees). This allowed for proportional adjustment of the servo angle via the potentiometer.
- **Digital LED Control:** The mapped servo angle was used to implement a threshold-based control logic for an external LED. The LED was programmed to be HIGH when the servo angle exceeded a predefined setpoint (e.g., 100), and LOW (extinguished) otherwise.

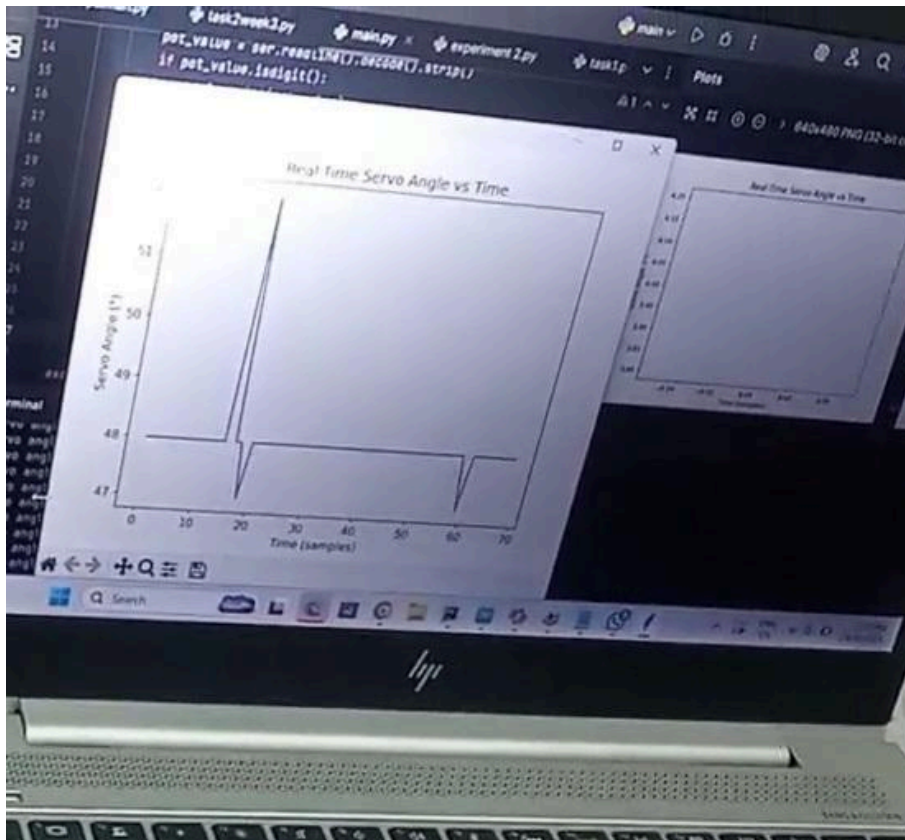
otherwise. This established an immediate, binary visual feedback mechanism confirming a specific control state.

The Arduino simultaneously transmitted the current mapped servo position over the serial port at a baud rate of 9600.

The following steps outline the host-side data reception and visualization procedure:

- A Python script utilizing the pyserial library was deployed to establish communication and continuously read the position data stream transmitted by the Arduino.
- The script was configured with a keyboard interrupt handler to allow for clean termination of the serial connection.
- For real-time graphical visualization, the script integrated the matplotlib library, operating in interactive mode (`plt.ion()`).
- Received data was dynamically plotted against time, providing a continuous graphical record of the servo's angle changes as the potentiometer was manipulated.

This finalized system successfully demonstrated the application of a serial interface for simultaneous physical control, digital state feedback, and dynamic data logging.



8.0 DISCUSSION

The expected functional relationship between the potentiometer input and the system outputs (LED brightness and servo motor position) was successfully demonstrated. Proportional changes in both outputs were achieved corresponding to adjustments of the potentiometer. However, minor discrepancies were noted, including intermittent flickering in the LED, a slight delay in servo response, and instances of non-linear behavior.

Sources of Error and Limitations:

1. Electrical Noise: Fluctuations in Analog-to-Digital Converter readings were caused by electrical noise, which consequently affected the stability of the LED brightness and the servo motor.
2. Hardware Tolerances: Minor deviations in the system output were introduced due to inherent variations in the potentiometer's resistance and the servo motor's inherent accuracy.
3. Non-Linear Response: A perfectly proportional relationship was not achieved between the physical potentiometer adjustment and the human perception of LED brightness or the observed servo movement.
4. Limited Range: The full analog input range was not consistently utilized by the potentiometer across its entire mechanical rotation.

Despite these limitations, the experiment effectively showcased real-time control using PWM signals.

9.0 CONCLUSION

The objective of the experiment, demonstrating the bi-directional utility of serial communication, was successfully met through the control of both LED brightness and servo motor position based on potentiometer data interpreted by a Python environment. The results confirmed that serial communication serves as an efficient and reliable protocol for real-time data transmission between a microcontroller and a host system. Crucially, the experiments validated that the data derived from the potentiometer's resistance adjustment could be directly translated into control commands for physical outputs. These principles of seamless sensor-to-actuator control, mediated by serial data transfer, are fundamental to complex systems, such as Robotic Joint Control, where potentiometer-like sensors are used to provide the central controller with real-time angular position feedback (Cognifront, 2025). Furthermore, this architecture is directly applicable to Human-Machine Interfaces (HMI) in systems like flight simulators and audio mixing boards, where physical knob movements are digitized and used to control complex software parameters. The knowledge gained regarding the integration of sensors, microcontrollers, and actuators provides a strong foundation for designing and implementing advanced control systems in robotics and automated processes.

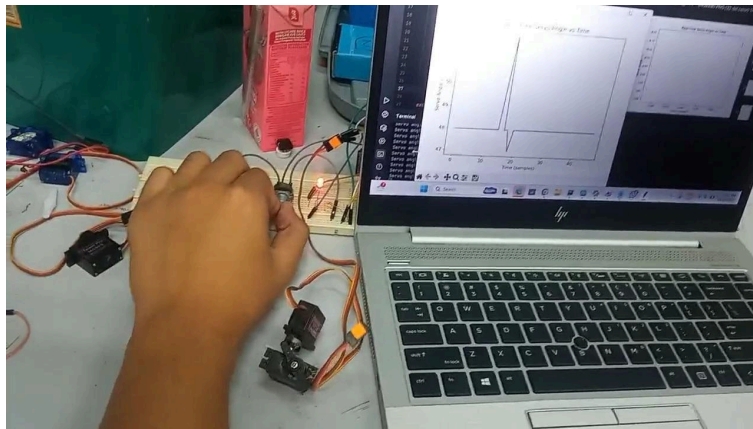
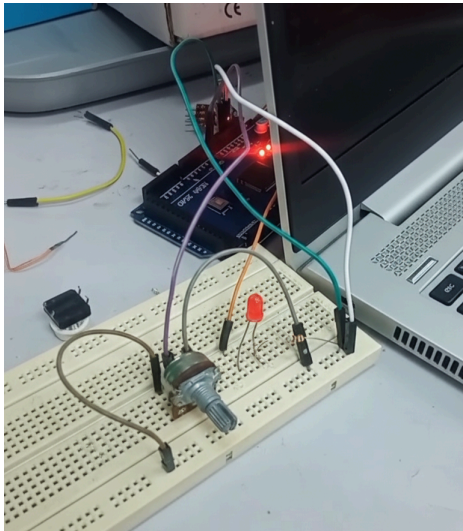
10.0 RECOMMENDATIONS

Following the results of this experiment, it is recommended that the following improvements be considered for future iterations of this control system. First, to address the issue of signal instability and LED flickering observed in the results, hardware-level electrical noise filtering should be implemented. This can be achieved by adding small capacitors across the power and ground terminals of the potentiometer, which is expected to smooth the analog input voltage and provide more stable ADC readings. Second, to improve the accuracy of the servo motor's positioning and correct the non-linear response, the existing linear (map) function should be replaced with a customized software calibration routine. This routine would employ a non-linear or polynomial equation within the Python script to precisely match the potentiometer's rotational input to the required angular output, thus ensuring a proportional and reliable response across the entire operating range.

11.0 REFERENCES

1. Pereyras, S. (2023, August). *Lab report: Potentiometer controlled LEDs*. Medium. <https://medium.com/@pereyras110/lab-report-potentiometer-controlled-leds-b34e06222416>
2. Arduino Forum. (2020, February). *Control servo motor via Python GUI (PyQt) component QDial*. Arduino Community Forum. <https://forum.arduino.cc/t/control-servo-motor-via-python-gui-pyqt-component-qdial/663997>
3. Matplotlib Developers. (n.d.). *Matplotlib: Visualization with Python*. Matplotlib Official Website. <https://matplotlib.org/>
4. Cognifront. (2025). *Experiment 3: Interfacing Potentiometer for Position Control Simulation*. https://www.cognifront.com/docs/index.php/Experiment_3:_Interfacing_Potentiometer_for_Position_Control_Simulation

12.0 APPENDICES



ACKNOWLEDGEMENTS

Special thanks to Zulkifli Bin Zainal Abidin & Wahyu Sediono for their guidance and support during this experiment. We would also like to extend our gratitude to our instructors, peers, and the laboratory staff for their valuable insights and assistance throughout the experiment. Their feedback and encouragement have greatly contributed to the success of this project. Finally, we appreciate the resources and learning materials provided, which enabled us to understand and implement serial communication effectively.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been undertaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual and all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read and understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report. We therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature:

Name: ainin sofiya binti kamaruzaman

Matric Number: 2314916

Signature: *iman*

Name: Muhammad Iman Farihin bin Mohd Imran

Matric Number: 2313813

Signature: *Zau*

Name: Muhammad Syazani Akmal Bin Mohd Nasarudin

Matric Number: 23134517

note week4b satt

<https://chatgpt.com/c/67de4805-7588-8013-b4bd-b1bad65b462e>

Interfacing the Cytron Technologies RFID-IDR-232N with an Arduino requires careful handling due to its RS232 communication protocol, which operates at voltage levels that can damage the Arduino's TTL (Transistor-Transistor Logic) inputs. Here's how you can safely connect and read data from the RFID reader:

1. Understanding the RFID-IDR-232N:

- **Communication Protocol:** RS232
- **Power Supply:** 5V via USB connection
- **Data Output:** 12-byte sequence per RFID scan, including start of heading, RFID ID, and start of text
- **Reading Range:** Approximately 2 cm
- **Response Time:** 0.1s

2. Voltage Level Conversion: RS232 signals can range between $\pm 12V$, which are incompatible with the Arduino's 0-5V logic levels. To interface the two safely, you need an RS232 to TTL converter, such as the MAX232 chip, which will shift the voltage levels to those suitable for the Arduino.

3. Hardware Connections:

- **RS232 to TTL Converter:**
 - Connect the RFID reader's RS232 output to the RS232 input of the converter.
 - Connect the converter's TTL output to the Arduino's RX (digital pin 0) for serial communication.
- **Power Supply:**
 - Connect the RFID reader's USB cable to a USB power source to provide the necessary 5V.
- **Common Ground:**
 - Ensure all components share a common ground to prevent communication issues.

4. Arduino Code Example:

```
#include <SoftwareSerial.h>
```

```
SoftwareSerial rfidSerial(2, 3); // RX, TX
```

```
void setup() {  
  Serial.begin(9600); // Initialize hardware serial for monitoring  
  rfidSerial.begin(9600); // Initialize software serial for RFID reader  
}
```

```
void loop() {
```

```

if (rfidSerial.available()) {
  String rfidData = "";
  while (rfidSerial.available()) {
    char incomingByte = rfidSerial.read();
    rfidData += String(incomingByte, HEX) + " ";
  }
  Serial.println("RFID Data: " + rfidData);
}
}

```

Explanation:

- A `SoftwareSerial` object is created to handle serial communication on pins 2 (RX) and 3 (TX).
- Both the hardware and software serial ports are initialized at a baud rate of 9600.
- The code checks if data is available from the RFID reader and reads it byte by byte, converting each byte to its hexadecimal representation for display.

5. Testing and Debugging:

- Upload the code to the Arduino.
- Open the Serial Monitor set to 9600 baud.
- Place an RFID tag near the reader.
- Observe the output in the Serial Monitor; you should see the RFID data in hexadecimal format.

6. Important Considerations:

- **Voltage Compatibility:** Always use an RS232 to TTL converter to protect the Arduino from high voltage levels.
- **Reading Distance:** The RFID-IDR-232N has a short reading range (~2 cm), so ensure the tag is placed close enough to be read.
- **Data Parsing:** The RFID reader sends a 12-byte sequence per scan. You may need to parse this data to extract the actual RFID tag ID, depending on your application's requirements.

By following these steps, you can safely and effectively interface the Cytron Technologies RFID-IDR-232N with an Arduino for your RFID-based projects.

draft

Conclusion	- Summarize the finding and their significance This experiment successfully demonstrated two serial communication applications of data transmission from a potentiometer and servo motor control via Python. The experiments provide a foundation for further automation and robotics application - Restate whether experiment supported or contradicted the hypothesis. The experiment confirms that serial communication efficiently transmits real-time data which the Arduino can read from the potentiometer and send to Python by the USB connection. Besides, in experiment 3b, it is valid, showing the potentiometer can control the movement of the servo accordingly. This confirms that serial communication can not only transfer data but also send commands that control physical devices. - Offer insights into the broader applications or implications of the experiment's results This kind of fundamental is crucial to have strong understanding for facing real-life industry application in industry automation, IoT projects and real-time monitoring systems.
-------------------	---

recomendations	For upskil, and improving the result, we could - Enhance the real-time data visualization which can be apply

9. Conclusion :	
•	Summarize the main findings and their significance.
•	Restate whether the experiment supported or contradicted the hypothesis.
•	Offer insights into the broader applications or implications of the experiment's results.
10. Recommen- dations:	
•	Suggest improvements or modifications for future iterations of the experiment.
•	Share any insights or lessons learned that might benefit future students.

RECOMMENDATIONS

- **Enhancing real-time data visualization:** Implementing real-time graphical visualization of the potentiometer readings and servo movements can provide a clearer understanding of how inputs and outputs change over time. This can be done using `matplotlib` for dynamic plotting or integrating real-time dashboards.
- **Implementing a graphical user interface (GUI):** A GUI can improve user interaction by allowing users to control the servo motor with sliders instead of manually entering angle values. Libraries like Tkinter or PyQt can be used to develop an intuitive interface for controlling and monitoring the system.
- **Expanding the experiment with additional actuators:** Future experiments could integrate other actuators like stepper motors, DC motors, or robotic arms to explore more advanced automation techniques. Adding more input devices such as sensors could also enhance the system's functionality and allow for more complex control mechanisms.
- **Lessons learned:** Ensuring proper serial port selection and matching baud rates is crucial for reliable communication. Debugging serial communication issues early can save significant time during implementation.
- **Future modifications:** Incorporating wireless communication (e.g., Bluetooth, Wi-Fi) to replace the USB cable connection and expanding the experiment with feedback control for closed-loop automation.